

Scalable NLP Pipeline for MENA Media Content

Contents

1	Introduction	3
2	Architecture Overview	3
2.1	Design Philosophy	3
2.2	Project Structure	3
2.3	Data Flow	4
3	Database Schema	4
3.1	Result Tables	4
3.2	Schema Details	5
3.2.1	Language Detection Table	5
3.2.2	NER Results Table	5
3.2.3	Sentiment Results Table	5
3.2.4	Keyword Results Table	6
4	Preprocessing Module	6
4.1	Preprocessing Operations	6
4.2	Task-Specific Presets	7
4.3	Design Rationale	7
5	Language Detection	7
5.1	Model	7
5.2	Process	8
5.3	Threshold Filtering	8
6	Named Entity Recognition (NER)	8
6.1	Models Compared	8
6.2	Label Unification	8
6.3	Technical Challenges and Solutions	9
6.3.1	Token-Based Chunking	9
6.3.2	Word-Boundary Expansion	9
6.3.3	Deduplication	9
6.3.4	Adjacent Entity Merging	9
6.4	Output Format	9

7	Sentiment Analysis	9
7.1	Models Compared	9
7.2	Probability Normalization	10
7.3	Chunking and Aggregation	10
7.4	Output Format	10
8	Keyword Extraction	10
8.1	Method	10
8.2	TF-IDF Formula	11
8.3	Parameters	11
8.4	Process	11
8.5	Output Format	11
9	Experiment Management	12
9.1	Run Configuration	12
9.2	Logging	12
9.3	Table Reset Strategy	12
10	Technology Stack	12
11	Conclusion	13

1 Introduction

This document presents the technical architecture and implementation details of a scalable Natural Language Processing (NLP) pipeline designed to analyze Arabic press and social media articles originating from the MENA region (specifically, a Libyan dataset). The pipeline processes millions of articles through four core NLP tasks:

1. **Language Detection** — identifying the language of each article.
2. **Named Entity Recognition (NER)** — extracting persons, locations, organizations, and events.
3. **Sentiment Analysis** — classifying article polarity (positive, negative, neutral).
4. **Keyword Extraction** — identifying the most representative terms per article.

The system is built with Python 3.11 and leverages MySQL (via SQLAlchemy) for persistent storage, Hugging Face Transformers for deep learning inference, and fastText for lightweight language identification.

2 Architecture Overview

2.1 Design Philosophy

The pipeline follows a **modular clean architecture** approach with three guiding principles:

- **Read-only source data:** The raw article database (MySQL) is never modified. All NLP results are stored in dedicated result tables that are dropped and recreated at each pipeline execution.
- **Experiment traceability:** Every execution creates a unique timestamped folder under `experiments/` containing a detailed `run.log` and a `run_config.json` file that captures all parameters, model versions, and environment metadata for full reproducibility.
- **Task-specific preprocessing:** Rather than applying a single preprocessing function to all tasks, the pipeline defines specialized presets tailored to the linguistic requirements of each downstream model.

2.2 Project Structure

```
NLP_MENA /
|-- main.py           # Pipeline orchestrator
|-- src/
|   |-- config.py     # Centralized configuration
|   |-- db_config.py  # Database connection + table
|                   management
|   |-- preprocessing.py # Arabic text preprocessing
|   |-- language_detection.py # fastText language detector
```

```

|   |-- ner_extraction.py           # Transformer-based NER
|   |-- sentiment_analysis.py      # Transformer-based sentiment
|   |-- keyword_extraction.py      # TF-IDF keyword extractor
|-- models/
|   |-- lid.176.ftz                # fastText language model
|-- experiments/
|   |-- 2025-06-14_15-30-00/
|       |-- run.log
|       |-- run_config.json
|-- .env                           # Database credentials

```

Listing 1: Directory layout

2.3 Data Flow

The pipeline executes in the following sequential order:

- Step 1: Sampling** — Fetch N articles from the raw MySQL table with filtering constraints (non-null body, minimum length, valid language ID).
- Step 2: Preprocessing** — Apply task-specific text cleaning to produce four variants: `text_langdetect`, `text_ner`, `text_sentiment`, and `text_keywords`.
- Step 3: Language Detection** — Run fastText on `text_langdetect` and store results. Filter for Arabic articles with confidence $\geq$ threshold.
- Step 4: NER Inference** — Run three NER models on `text_ner` and store consolidated results (one row per article).
- Step 5: Sentiment Inference** — Run three sentiment models on `text_sentiment` and store unified results (one row per article).
- Step 6: Keyword Extraction** — Fit TF-IDF on the Arabic corpus and extract top- N keywords per article.

3 Database Schema

The pipeline manages seven result tables, all dropped and recreated at the beginning of each run to ensure clean state. The raw article table remains untouched.

3.1 Result Tables

Table Name	Format	Description
<code>lang_detection</code>	1 row per article	Language prediction, confidence, expected language, correctness flag
<code>preprocess_ner</code>	1 row per article	Preprocessed text for NER task
<code>preprocess_sentiment</code>	1 row per article	Preprocessed text for sentiment task
<code>preprocess_keywords</code>	1 row per article	Preprocessed text for keyword task

Table Name	Format	Description
ner_results	1 row per article	Entity strings from 3 models (AraBERT, CAMEL, mBERT)
sentiment_results	1 row per article	Label + score + probability JSON for each of 3 models
keyword_results	1 row per article	Ranked keyword list with TF-IDF scores

Table 1: Summary of the seven result tables.

3.2 Schema Details

3.2.1 Language Detection Table

```
CREATE TABLE lang_detection (
  article_id BIGINT PRIMARY KEY,
  predicted_lang VARCHAR(10),
  confidence FLOAT,
  expected_lang VARCHAR(10),
  is_correct BOOLEAN
);
```

Listing 2: lang_detection schema

3.2.2 NER Results Table

```
CREATE TABLE ner_results (
  article_id BIGINT PRIMARY KEY,
  arabert_entities LONGTEXT,
  camel_entities LONGTEXT,
  mbert_entities LONGTEXT
);
```

Listing 3: ner_results schema

Each entity column stores a formatted string following the pattern:

entity_text (LBL, score), entity_text (LBL, score), ...

Example:

```
, 0.99) (ORG, 0.99), . (LOC
```

3.2.3 Sentiment Results Table

```
CREATE TABLE sentiment_results (
  article_id BIGINT PRIMARY KEY,
  arabert_label VARCHAR(10),
  arabert_score FLOAT,
```

```

    arabert_probs JSON ,
    camel_label   VARCHAR(10) ,
    camel_score   FLOAT ,
    camel_probs   JSON ,
    mbert_label   VARCHAR(10) ,
    mbert_score   FLOAT ,
    mbert_probs   JSON
);

```

Listing 4: sentiment_results schema

3.2.4 Keyword Results Table

```

CREATE TABLE keyword_results (
    article_id BIGINT PRIMARY KEY ,
    keywords   LONGTEXT
);

```

Listing 5: keyword_results schema

The `keywords` column stores a ranked comma-separated string:

word1 (0.4134), word2 (0.2756), word3 (0.2756), ...

4 Preprocessing Module

Arabic text presents unique preprocessing challenges due to its morphological richness, diacritical marks, and encoding variations. The pipeline implements a centralized `ArabicPreprocessor` class with configurable parameters, and defines four task-specific presets.

4.1 Preprocessing Operations

Operation	Description
HTML cleaning	Remove HTML tags and decode entities (<code>&nbsp;</code> , <code>&amp;</code> , etc.)
NFC normalization	Standardize Unicode encoding (applied globally)
URL/email removal	Strip hyperlinks and email addresses
Tatweel removal	Remove the Arabic Kashida character (<code>`</code>) used for text stretching
Diacritics removal	Strip tashkīl marks (U+064B–U+065F)
Special character removal	Keep only Arabic characters, numbers, and Arabic punctuation
Number removal	Remove both Latin (0–9) and Arabic-Indic digits
Repeated character reduction	Collapse sequences like <code>to</code> to <code>to</code>
Hashtag handling	Replace <code>#</code> and <code>_</code> with spaces
Merged word fix	Insert space before concatenated (e.g., <code>→</code>)

Operation	Description
Whitespace normalization	Collapse multiple spaces into one; trim

Table 2: Available preprocessing operations.

4.2 Task-Specific Presets

Operation	Lang Detect	NER	Sentiment	Keywords
Diacritics	×			
Normalize Arabic				
URLs				
Emails				
Numbers	×	×	×	
Special chars	×			
Repeated chars	×	×	×	
Tatweel	×			
Hashtags	×			×
Fix merged words	×			

Table 3: Preprocessing presets per task. = enabled, × = disabled.

4.3 Design Rationale

- **Language Detection** uses minimal cleaning to preserve the original text distribution that fastText was trained on.
- **NER and Sentiment** apply intensive cleaning but deliberately preserve Alef forms (, ,) and Alef Maqsura () because BERT-based models rely on these distinctions for contextual understanding.
- **Keyword Extraction** applies the most aggressive normalization (including number removal and repeated character reduction) because TF-IDF operates on surface-level token frequency where orthographic variants would split the counts of identical concepts.

5 Language Detection

5.1 Model

Language detection uses the **fastText** library with the pre-trained `lid.176.ftz` model, which supports 176 languages. The model is loaded from the local `models/` directory.

5.2 Process

1. Each article’s body is preprocessed using the language detection preset.
2. The fastText model predicts a language code (ISO 639-1) and a confidence score.
3. The prediction is compared against the `id_language` field from the source database (mapped to a language code via a reference table) to compute an `is_correct` flag.
4. Results are stored in the `lang_detection` table.
5. Articles identified as Arabic (`lang = "ar"`) with confidence $\geq$ the threshold (default: 0.60) are passed to downstream NLP tasks.

5.3 Threshold Filtering

The language confidence threshold serves as a quality gate. Articles below the threshold are excluded from NER, sentiment, and keyword analysis to prevent noisy or non-Arabic text from degrading model performance.

6 Named Entity Recognition (NER)

6.1 Models Compared

The pipeline runs three NER models in parallel on each article:

ID	Model	Architecture
0	hatmimoha/arabic-ner	AraBERT fine-tuned
1	CAMEL-Lab/bert-base-arabic-camel6bert-base-ner	CAMEL-BERT MSA
2	Davlan/bert-base-multilingual-cased-ner	Multilingual BERT

Table 4: NER models used in the pipeline.

6.2 Label Unification

Different models produce different label schemes. The pipeline unifies all labels to **three-letter uppercase codes**:

Raw Label(s)	Unified
PERSON, PERS, PER	PER
LOCATION, LOC	LOC
ORGANIZATION, ORG	ORG
EVENT, EVE	EVE
MISCELLANEOUS, MISC	MIS
Any unknown label	First 3 letters

Table 5: NER label unification mapping.

6.3 Technical Challenges and Solutions

6.3.1 Token-Based Chunking

BERT models have a maximum input length of 512 tokens. Rather than naively truncating text at the character level (which can split words mid-token), the pipeline uses the tokenizer’s `offset_mapping` to create chunks that respect token boundaries. Each chunk overlaps with the previous one by a configurable number of tokens (default: 100) to ensure entities near chunk boundaries are not missed.

6.3.2 Word-Boundary Expansion

When the model identifies a short entity (e.g., instead of), the pipeline expands the entity span to the nearest whitespace or punctuation boundary. This is controlled by the `expand_short_entities` parameter (default: enabled for entities ≤ 3 characters).

6.3.3 Deduplication

Overlapping chunks can produce duplicate entity detections. The pipeline deduplicates by `(start, end, label)`, keeping the detection with the highest confidence score.

6.3.4 Adjacent Entity Merging

If two consecutive entities share the same label and are separated only by whitespace or light punctuation (e.g., commas, periods), they are merged into a single entity. The merged entity takes the maximum confidence score of its components.

6.4 Output Format

Results are stored as one row per article with three columns, each containing a formatted entity string:

```
article_id | arabert_entities | camel_entities | mbert_entities
216385198 |                  (ORG, 0.99),                  (LOC,
0.99) | ... | ...
```

7 Sentiment Analysis

7.1 Models Compared

ID	Model	Architecture	Native Labels
0	PRALI22/AraBert-Arabic-SentimentAnalysis	AraBERT	POS/NEG/NEU/MIX
1	CAMEL-Lab/bert-base-arabic-camel-bert-sentiment	CamBERT	POS/NEG/NEU
2	nlptown/bert-base-multilingual-mixed-sentiment	multilingual-BERT	1–5 stars

Table 6: Sentiment models used in the pipeline.

7.2 Probability Normalization

Each model produces a different output schema. The pipeline applies a **probability normalizer** function to map all outputs into a unified space:

- **AraBERT (PRAli22)**: Maps `Positive/Negative/Neutral/Mixed` → `POS/NEG/NEU/MIX`. The `MIX` class is preserved in the probability distribution.
- **CAMeL**: Maps `positive/negative/neutral` → `POS/NEG/NEU`.
- **mBERT (nlptown)**: Converts a 5-star rating into 3 classes:

$$\text{NEG} = P(1 \text{ star}) + P(2 \text{ stars})$$

$$\text{NEU} = P(3 \text{ stars})$$

$$\text{POS} = P(4 \text{ stars}) + P(5 \text{ stars})$$

7.3 Chunking and Aggregation

Like NER, sentiment analysis must handle texts exceeding the 512-token limit. The pipeline supports two aggregation strategies:

- **Mean probabilities (default)**: Each chunk produces a probability distribution. The final distribution is a weighted average across chunks, weighted by chunk length in tokens. This ensures longer chunks contribute proportionally more.
- **Max chunk**: The chunk with the highest maximum probability determines the final prediction.

7.4 Output Format

Each article produces one row with nine columns (three models × three fields):

<code>article_id</code>	<code> </code>	<code>arabert_label</code>	<code> </code>	<code>arabert_score</code>	<code> </code>	<code>arabert_probs</code>
		<code> camel_label</code>		<code> camel_score</code>		<code> camel_probs</code>
		<code> mbert_label</code>		<code> mbert_score</code>		<code> mbert_probs</code>

All scores are rounded to 4 decimal places. The `probs` columns store full JSON probability distributions, e.g.:

```
{"POS": 0.0177, "NEG": 0.0071, "NEU": 0.9718, "MIX": 0.0033}
```

8 Keyword Extraction

8.1 Method

Keyword extraction uses **TF-IDF** (Term Frequency–Inverse Document Frequency) implemented via scikit-learn’s `TfidfVectorizer`. Unlike the deep learning models used for NER and sentiment, TF-IDF is a statistical method that measures the importance of a term relative to a corpus.

8.2 TF-IDF Formula

The TF-IDF weight of a term t in a document d within a corpus D is:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D) \quad (1)$$

where:

$$\text{TF}(t, d) = \frac{\text{count of } t \text{ in } d}{\text{total terms in } d} \quad (2)$$

$$\text{IDF}(t, D) = \log \frac{|D|}{|\{d \in D : t \in d\}|} \quad (3)$$

8.3 Parameters

Parameter	Default	Description
max_features	1000	Maximum vocabulary size
top_n	10	Number of keywords extracted per article
max_df	0.95	Ignore terms appearing in >95% of documents
min_df	1	Minimum document frequency for a term
Stopwords	72 words	Arabic stopwords defined in <code>Config</code>

Table 7: TF-IDF parameters.

8.4 Process

1. The vectorizer is **fitted** on the entire Arabic corpus (all articles passing the language filter in the current run).
2. For each article, the vectorizer **transforms** the text into a sparse TF-IDF vector.
3. The top N features (by score) are extracted as keywords.
4. Results are stored as a single comma-separated string per article.

8.5 Output Format

article_id		keywords
216385198		(0.4134), (0.2756), (0.2756), ...

9 Experiment Management

9.1 Run Configuration

Every pipeline execution generates a `run_config.json` file that captures:

- **Environment:** Python version, OS platform, and exact versions of all dependencies (pandas, SQLAlchemy, transformers, torch, fastText, scikit-learn).
- **Database:** Host, port, database name, and source table.
- **Sampling:** Sample size, SQL filters, and ordering.
- **Preprocessing:** Complete parameter dictionaries for all four presets.
- **Models:** Model identifiers and Hugging Face repository names.
- **Task parameters:** All thresholds, chunk sizes, overlap sizes, and aggregation strategies.

This ensures that any experiment can be fully reproduced by re-running the pipeline with the same configuration file.

9.2 Logging

The `run.log` file records:

- Timestamped entries for every processing step.
- Per-article results for language detection, NER entity counts, and sentiment labels.
- Error messages for any model inference failures (with graceful degradation — the pipeline continues processing remaining articles).

9.3 Table Reset Strategy

At the beginning of each run, the pipeline:

1. Drops all seven result tables if they exist from a previous run.
2. Recreates them with fresh schemas.
3. Inserts new results row by row.

This guarantees that the database always reflects the most recent execution and avoids accumulation of stale data.

10 Technology Stack

Component	Technology	Role
Language	Python 3.11	Core programming language
Database	MySQL + SQLAlchemy	Persistent storage with connection pooling
NER / Sentiment	Hugging Face Transformers	Pre-trained BERT model inference
Language Detection	fastText (lid.176.ftz)	Lightweight 176-language classifier
Keywords	scikit-learn TF-IDF	Statistical keyword importance
Deep Learning	PyTorch	Backend for Transformers models
Configuration	python-dotenv	Environment variable management

Table 8: Technology stack summary.

11 Conclusion

This pipeline provides a complete, reproducible, and modular framework for analyzing Arabic media content at scale. Key design strengths include:

- **Task-specific preprocessing** that respects the linguistic requirements of each downstream model, avoiding both under-processing (noisy input) and over-normalization (loss of meaningful distinctions).
- **Multi-model comparison** for NER and sentiment, enabling cross-validation of results and identification of model-specific biases.
- **Robust text chunking** that handles arbitrarily long documents without truncation artifacts.
- **Unified output schemas** that normalize heterogeneous model outputs into comparable formats.
- **Full reproducibility** through comprehensive experiment logging and configuration capture.

The current implementation stores results in MySQL with a drop-and-recreate strategy per run. Future enhancements could include run-level versioning (preserving results across runs for longitudinal comparison), batch processing optimizations for million-scale datasets, and a REST API layer for real-time inference.